© Guven

Sound Generation

Example MFCC

Mel-Frequency Cepstral Coefficients (MFCCs) are
features commonly used in speech and audio processing to represent a signal's spectral characteristics. They are derived from the Mel scale, which mimics human auditory perception by emphasizing lower frequencies. The process of computing MFCCs involves taking the Fourier transform of a signal, applying a Mel filter bank, and then computing the discrete cosine transform (DCT) to obtain a compact representation. MFCCs are widely used in applications such as speech recognition, speaker identification, and audio classification because they effectively capture timbral features of sound.

The following dataset contains audio files in WAV format, which can be read by the `torchaudio` backend and the `soundfile` library.

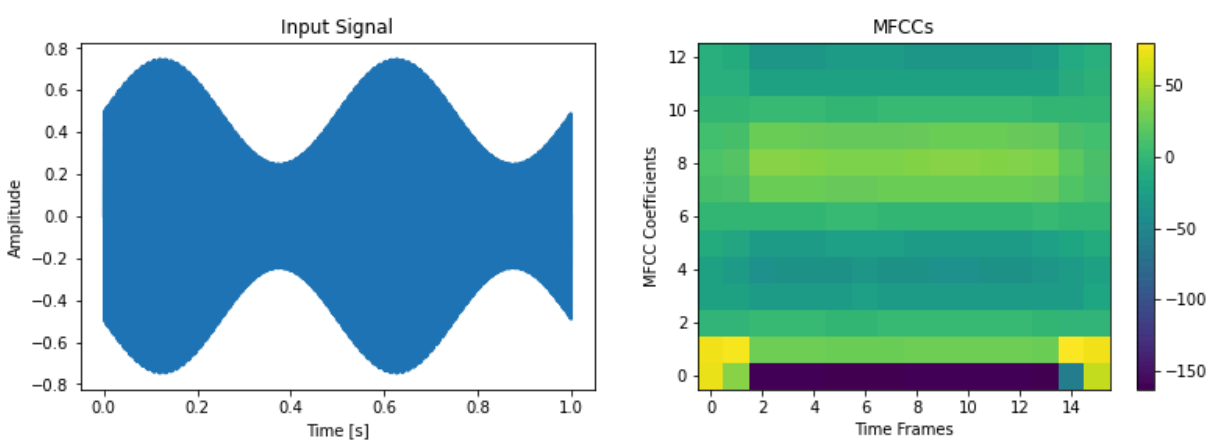
`pip install soundfile` ↩

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 72
import numpy as np

import torch
import torchaudio
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

Device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

print(f'PyTorch version= {torch.__version__}')
print(f'torchaudio version= {torchaudio.__version__}')
print(f'CUDA available= {torch.cuda.is_available()}')
print(str(torchaudio.list_audio_backends()))  # cant be empty

PATH_DATA = '/EP_datasets/cats_dogs/'
```

```
PyTorch version= 2.5.1
torchaudio version= 2.5.1
CUDA available= True
['soundfile']
```

In [2]:
```
# Sample rate and signal
sample_rate = 8000
```

```
duration = 1  # seconds
t = torch.linspace(0, duration, int(sample_rate * duration))
frequency = 440  # A4 note
signal = 0.5 * torch.sin(2 * torch.pi * frequency * t)

# Add a simple amplitude modulation
modulator = 1.0 + 0.5 * torch.sin(2*torch.pi *2*t)
signal = signal*modulator

fig, ax = plt.subplots(1, 2, figsize=(13,4), dpi=72)
ax[0].plot(t.numpy(), signal.numpy())  # plot the signal
ax[0].set(xlabel='Time [s]', ylabel='Amplitude', title='Input Signal')

# Compute MFCCs
n_mfcc = 13
mfcc_transform = torchaudio.transforms.MFCC(
    sample_rate=sample_rate,
    n_mfcc=n_mfcc,
    melkwargs={
        'n_fft': 2048,
        'n_mels': 40,
        'hop_length': 512,
        'mel_scale': 'htk',
    }
)

mfcc = mfcc_transform(signal.unsqueeze(0))

cax1 = ax[1].imshow(mfcc[0].numpy(), aspect='auto', origin='lower')  # plo
ax[1].set(xlabel='Time Frames', ylabel='MFCC Coefficients', title='MFCCs')
fig.colorbar(cax1)
plt.show()
```



Generative Sound Creation by WaveNet

WaveNet is a deep generative model for raw audio developed by DeepMind. It is a convolutional neural network (CNN) that synthesizes audio waveforms with high fidelity.

- Causal Convolutions ensures the model only uses past information to predict future samples
- Dilated Convolutions expands the receptive field exponentially, allowing the model to capture long-range dependencies efficiently
- Residual and Skip Connections help with gradient flow and improve convergence
  - Residual connections are shortcuts that add the input of a layer to its output, helping in training deep networks by mitigating the vanishing gradient problem
  - Each dilated convolutional layer's output is combined with the input using a residual connection, allowing information to flow directly through layers without significant loss
  - Skip connections allow intermediate layer outputs to bypass several layers and directly contribute to the final output
  - Each layer's processed signal is passed to the output layers via skip connections
  - This improves gradient propagation, speeds up training, and enhances the quality of generated audio
- Softmax Output produces a probability distribution over possible next sample values

WaveNet learns from real audio waveforms and generates new samples sequentially, producing highly realistic speech, music, and sound effects.

In [3]:
```
class WaveNet(nn.Module):
    def __init__(self, quantization_channels=256, residual_channels=32, sk
        super(WaveNet, self).__init__()

        self.quantization_channels = quantization_channels
        self.residual_channels = residual_channels
        self.skip_channels = skip_channels
        self.dilation_depth = dilation_depth
        self.filter_size = filter_size

        self.causal_conv = nn.Conv1d(
            in_channels=quantization_channels,
            out_channels=residual_channels,
            kernel_size=filter_size,
            padding=filter_size - 1
        )

        self.dilated_convs = nn.ModuleList()
        self.residual_convs = nn.ModuleList()
        self.skip_convs = nn.ModuleList()

        for i in range(dilation_depth):
            dilation = 2 ** i
            padding = (filter_size - 1) * dilation
```

```
            conv_filter = nn.Conv1d(
                in_channels=residual_channels,
                out_channels=residual_channels,
                kernel_size=filter_size,
                dilation=dilation,
                padding=padding
            )
            conv_gate = nn.Conv1d(
                in_channels=residual_channels,
                out_channels=residual_channels,
                kernel_size=filter_size,
                dilation=dilation,
                padding=padding
            )
            self.dilated_convs.append(nn.ModuleList([conv_filter, conv_gat

            self.residual_convs.append(nn.Conv1d(
                in_channels=residual_channels,
                out_channels=residual_channels,
                kernel_size=1
            ))
            self.skip_convs.append(nn.Conv1d(
                in_channels=residual_channels,
                out_channels=skip_channels,
                kernel_size=1
            ))

        self.relu = nn.ReLU()
        self.output_conv1 = nn.Conv1d(
            in_channels=skip_channels,
            out_channels=skip_channels,
            kernel_size=1
        )
        self.output_conv2 = nn.Conv1d(
            in_channels=skip_channels,
            out_channels=quantization_channels,
            kernel_size=1
        )

    def forward(self, x):
        # input x shape: (batch_size, seq_len)
        x = F.one_hot(x, num_classes=self.quantization_channels).float()
        # reshape x to (batch_size, channels, seq_len)
        x = x.transpose(1, 2)

        x = self.causal_conv(x)
        x = x[:, :, :- (self.filter_size - 1)]  # Remove padding to make c
        skip_connections = []

        for i in range(self.dilation_depth):
            conv_filter, conv_gate = self.dilated_convs[i]

            filter_out = conv_filter(x)
            gate_out = conv_gate(x)
```

```
            pad = (self.filter_size - 1) * (2 ** i)
            filter_out = filter_out[:, :, :-pad]
            gate_out = gate_out[:, :, :-pad]

            tanh_out = torch.tanh(filter_out)
            sigmoid_out = torch.sigmoid(gate_out)
            z = tanh_out * sigmoid_out

            skip = self.skip_convs[i](z)
            skip_connections.append(skip)

            res = self.residual_convs[i](z)
            x = x + res

        out = sum(skip_connections)
        out = self.relu(out)
        out = self.output_conv1(out)
        out = self.relu(out)
        out = self.output_conv2(out)

        out = out.mean(dim=2)

        out = F.softmax(out, dim=1)
        return out
```

In [4]:

```
import os

Flist = [os.path.join(PATH_DATA,f) for f in os.listdir(PATH_DATA) if f.low
print(f'#Files= {len(Flist)}')


def preprocess_audio(_path, q_channels=256):
    import wave
    with wave.open(_path, 'rb') as wf:
        # check if audio is mono and PCM encoded (standard format for Wave
        assert wf.getnchannels() == 1, 'Only mono audio is supported.'
        assert wf.getsampwidth() == 2, 'Only 16-bit PCM WAV files are supp

        # read raw PCM audio data
        frames = wf.readframes(wf.getnframes())
        audio = np.frombuffer(frames, dtype=np.int16).astype(np.float32)

    audio /= np.max(np.abs(audio))  # normalize to [-1, 1] range

    # μ-law encoding
    mu = q_channels - 1
    audio_mu = np.sign(audio) * np.log1p(mu*np.abs(audio)) / np.log1p(mu)
    quantized_audio = ((audio_mu + 1) / 2*mu).astype(np.int64)

    # convert to tensor with the shape (1, seq_length)
    audio_tensor = torch.tensor(quantized_audio, dtype=torch.long).unsquee
    return audio_tensor
```

```
# sanity
x = preprocess_audio(PATH_DATA+'cat_1.wav')
print(x.shape)
```

```
#Files= 164
torch.Size([1, 179200])
```

In [5]:
```
def pad_seq(_x, target_length):
    T = _x.size(1)
    if T < target_length:
        padding = target_length - T  # pad with zeros at the end
        x = torch.cat([_x, torch.zeros(1, padding, dtype=torch.long)], dim
    return x

def truncate_seq(_x, target_length):
    if _x.size(1) > target_length:
        x = _x[:, :target_length]
    return x

def split_into_chunks(_x, chunk_length):
    total_length = _x.size(1)
    num_chunks = total_length // chunk_length
    chunks = [_x[:, i * chunk_length: (i + 1) * chunk_length] for i in ran

    # handle remainder
    remainder = total_length % chunk_length
    if remainder:
        last_chunk = _x[:, -chunk_length:]  # take the last chunk with pad
        chunks.append(last_chunk)

    return chunks

# sanity
chunks = split_into_chunks(x, chunk_length=16000)
print(f"Total chunks: {len(chunks)}")
```

```
Total chunks: 12
```

In [6]:
```
class AudioDataset(Dataset):
    def __init__(self, audio_files, target_length=16000, q_channels=256):
        self.audio_files = audio_files
        self.target_length = target_length
        self.q_channels = q_channels

    def __len__(self):
        return len(self.audio_files)

    def __getitem__(self, idx):
        audio_tensor = preprocess_audio(self.audio_files[idx], self.q_chan

        # pad or truncate
        if audio_tensor.size(1) < self.target_length:
            audio_tensor = pad_seq(audio_tensor, target_length=self.target
        elif audio_tensor.size(1) > self.target_length:
            audio_tensor = truncate_seq(audio_tensor, target_length=self.t
```

```
        return audio_tensor.squeeze(0)  # remove batch dimension for DataL

dataset = AudioDataset(Flist)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

📚 References

1. Van Den Oord, Aaron, et al. "Wavenet: A generative model for raw audio." arXiv preprint arXiv:1609.03499 12 (2016).

In [7]:

```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```